Tarea 2 – Redes Neuronales con Keras: solución y guía

Oct 15, 2026 · @NATO

Resultados de la ejecución en Colab

La tarea quedó enviada a UNCode con nota de 100.0%: los 18 tests (`1_1_1` a `2_3_2`) respondieron "Test Run correctly" con nota 100 cada uno.

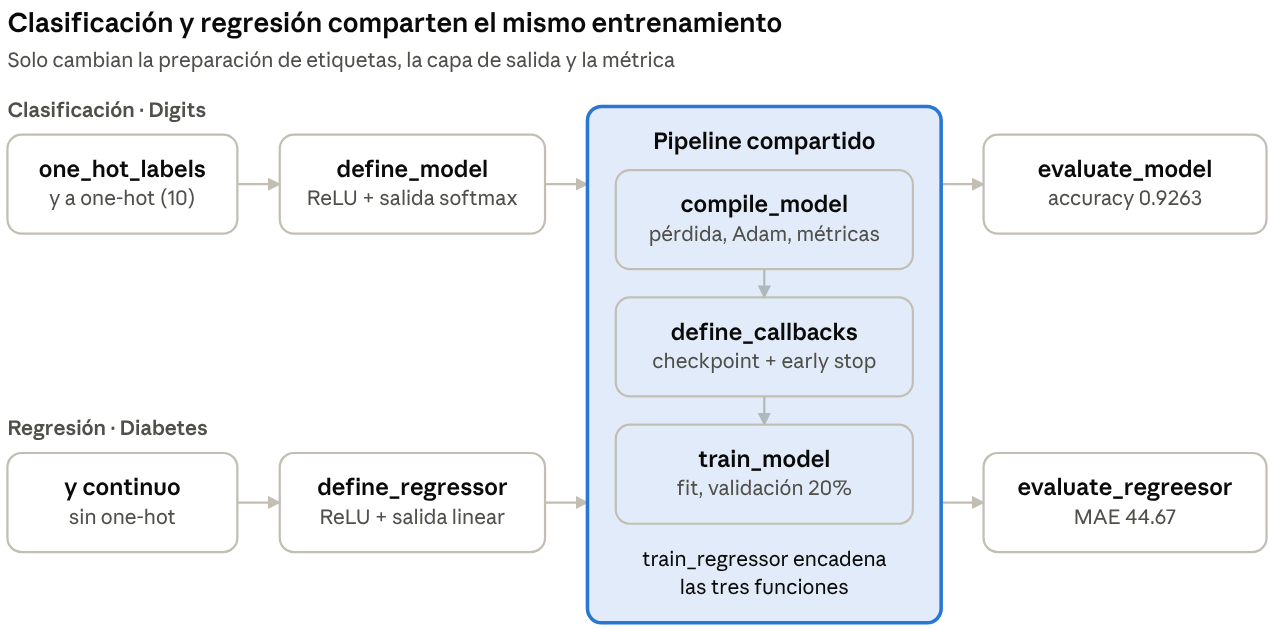
Sección	Función	Tests	Nota
1.1	`one_hot_labels`	1_1_1, 1_1_2	100 / 100
1.2	`define_model`	1_2_1, 1_2_2	100 / 100
1.3	`compile_model`	1_3_1, 1_3_2	100 / 100
1.4	`define_callbacks`	1_4_1, 1_4_2	100 / 100
1.5	`train_model`	1_5_1, 1_5_2	100 / 100
1.6	`evaluate_model`	1_6_1, 1_6_2	100 / 100
2.1	`define_regressor`	2_1_1, 2_1_2	100 / 100
2.2	`train_regressor`	2_2_1, 2_2_2	100 / 100
2.3	`evaluate_regreesor`	2_3_1, 2_3_2	100 / 100
Envío	`grader.submit_task`	todos	100.0%

Cada salida impresa en Colab coincide con la salida esperada del enunciado:

Celda de prueba	Esperado	Obtenido en Colab
One-hot de [0,1,0,0,0,0,1,2,1,2] y [0,1,2]	matrices 10×3 y 3×3	idénticas
`define_model(64, 2, 16, 10).summary()`	1,040 + 272 + 170 = 1,482 parámetros	1,482

Celda de prueba	Esperado	Obtenido en Colab
`get_compile_config()`	Adam, lr 0.0010000000474974513, loss `categorical_crossentropy`, metrics ['acc']	igual
`define_callbacks('test_checkpoint.weights.h5', 'loss', 'min', 2)`	monitor loss, save_best_only True, patience 2, restore True	igual
Entrenamiento 5 épocas: loss por época	4.02000 → 2.43602 → 2.06657 → 1.84849 → 1.65731	igual; restaura la época 5
Accuracy en test (5 épocas)	0.4186	0.4186
Entrenamiento 50 épocas (lr 1e-2)	early stopping, accuracy 0.9263	se detiene en la época 30, mejor val_loss 0.20765, accuracy 0.9263
`define_regressor(10, 2, 8, 1).summary()`	88 + 72 + 9 = 169 parámetros	169
Regresor, época 1 (primer batch)	loss 151.8743, val_loss 162.67799	igual
Regresor, final	mejor época 48, val_loss 46.27600	igual
MAE en test	44.6665	44.6665

Cómo encajan las funciones



funciones del notebook · 2 problemas, 1 pipeline

La tarea es incremental: `train_regressor` no reimplementa nada, llama a `compile_model`, `define_callbacks` y `train_model` de la parte 1. Por eso un error en la parte 1 rompe también los tests de la parte 2.

Parte 1: clasificación multiclase (Digits)

Digits tiene 1,797 imágenes de 8×8 = 64 píxeles y 10 clases. La red recibe un vector de 64, pasa por capas densas ReLU y termina en 10 neuronas softmax que dan una probabilidad por dígito.

1.1 one_hot_labels

```
def one_hot_labels(y_train, y_test):
    num_classes = int(max(np.max(y_train), np.max(y_test))) + 1
    y_train_ohe = keras.utils.to_categorical(y_train,
num_classes=num_classes)
    y_test_ohe = keras.utils.to_categorical(y_test, num_classes=num_classes)
    return y_train_ohe, y_test_ohe
```

- Softmax produce un vector de probabilidades, así que la etiqueta debe tener la misma forma: 2 → [0, 0, 1].

- Se calcula `num_classes` con el máximo de ambos conjuntos y se pasa explícito. Si no, un test con menos clases que train generaría matrices de ancho distinto.

1.2 define_model (API funcional)

```
def define_model(input_shape, hidden_layers, hidden_units, output_units):
    keras.utils.set_random_seed(0)
    inputs = keras.layers.Input(shape=(input_shape,))
    x = inputs
    for _ in range(hidden_layers):
        x = keras.layers.Dense(hidden_units, activation='relu')(x)
    outputs = keras.layers.Dense(output_units, activation='softmax')(x)
    model = keras.Model(inputs=inputs, outputs=outputs)
    return model
```

- API funcional: cada capa se "llama" sobre el tensor anterior (`Dense(...)(x)`) y al final se crea `keras.Model(inputs, outputs)` . Es lo que pide el enunciado en vez de `Sequential` .
- El `for` permite un número variable de capas ocultas reutilizando la variable `x` .
- Conteo de parámetros (entradas × neuronas + sesgos): 64×16+16 = 1,040; 16×16+16 = 272; 16×10+10 = 170; total 1,482.
- La semilla se fija antes de crear capas para que los pesos iniciales sean siempre los mismos (el autograder compara números).

1.3 compile_model

```
def compile_model(model, loss, optimizer, metrics):
    keras.utils.set_random_seed(0)
    model.compile(loss=loss, optimizer=optimizer, metrics=metrics)
    return model
```

Compilar une tres piezas: qué minimizar (pérdida), cómo actualizar pesos (optimizador Adam) y qué reportar (métricas). `model.compile` modifica el modelo en sitio; por eso se devuelve el mismo objeto.

1.4 define_callbacks

```
def define_callbacks(filepath, metric, mode, patience):
    checkpoint = keras.callbacks.ModelCheckpoint(filepath=filepath,
monitor=metric, mode=mode,
                                                 save_best_only=True,
save_weights_only=True, verbose=1)
    stopping = keras.callbacks.EarlyStopping(monitor=metric, mode=mode,
patience=patience,
                                             restore_best_weights=True,
verbose=1)
    return [checkpoint, stopping]
```

- `ModelCheckpoint` guarda en disco los pesos cada vez que la métrica mejora (`save_best_only=True`). Con `save_weights_only=True` el archivo debe terminar en `.weights.h5` (requisito de Keras 3).
- `EarlyStopping` corta el entrenamiento si la métrica no mejora en `patience` épocas. `restore_best_weights=True` cumple la nota del enunciado: los mejores pesos quedan cargados automáticamente en el modelo.
- `mode='min'` para pérdidas, `'max'` para accuracy. `verbose=1` imprime los mensajes "loss improved from…" de la salida esperada.

1.5 train_model

```
def train_model(model, X_train, y_ohe_train, epochs, batch_size,
validation_rate, callbacks):
    keras.utils.set_random_seed(0)
    history = model.fit(X_train, y_ohe_train, epochs=epochs,
batch_size=batch_size,
                        validation_split=validation_rate,
callbacks=callbacks)
    return history, model
```

- `validation_split=0.2` aparta el último 20% de las filas (sin barajar) como validación: 1,078 × 0.8 ≈ 862 muestras de entrenamiento → 14 batches de 64, como en la salida.
- `fit` devuelve un objeto `History`; `history.history` es un diccionario con `loss`, `acc`, `val_loss`, `val_acc` por época, que es lo que se grafica.

1.6 evaluate_model

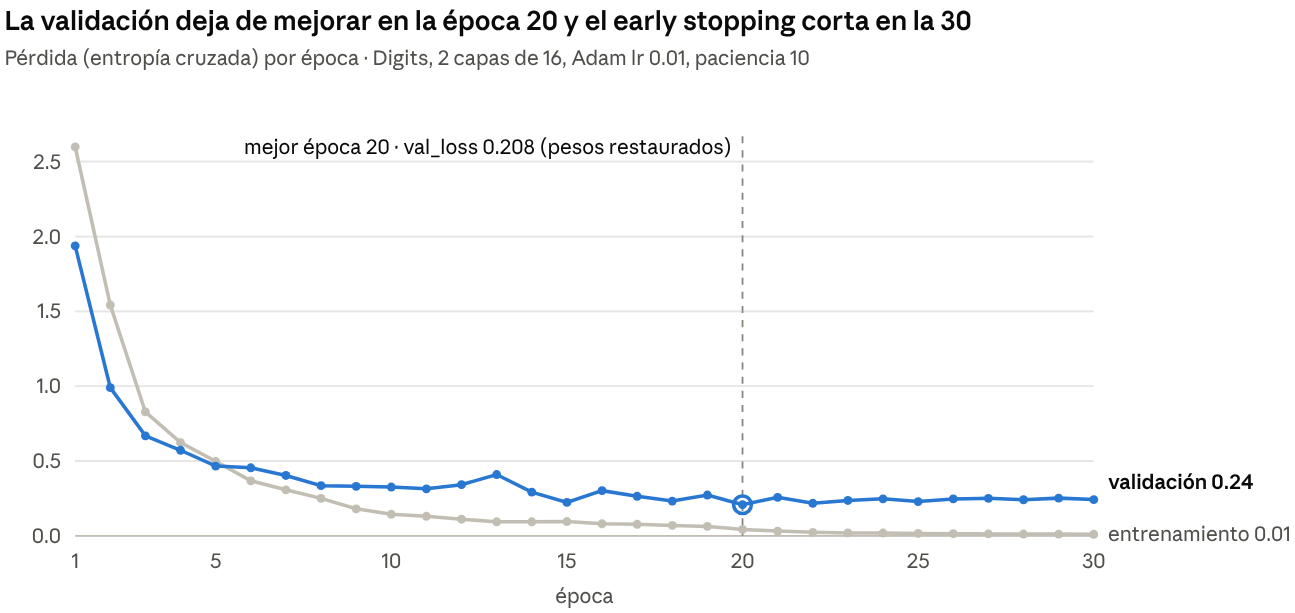
```
def evaluate_model(model, X_test, y_ohe_test):
    loss, acc = model.evaluate(X_test, y_ohe_test)
    return acc
```

`evaluate` devuelve `[pérdida, métrica_1, …]` en el orden de `metrics`. Con `metrics=['acc']` el segundo valor es el accuracy.

Entrenamiento largo

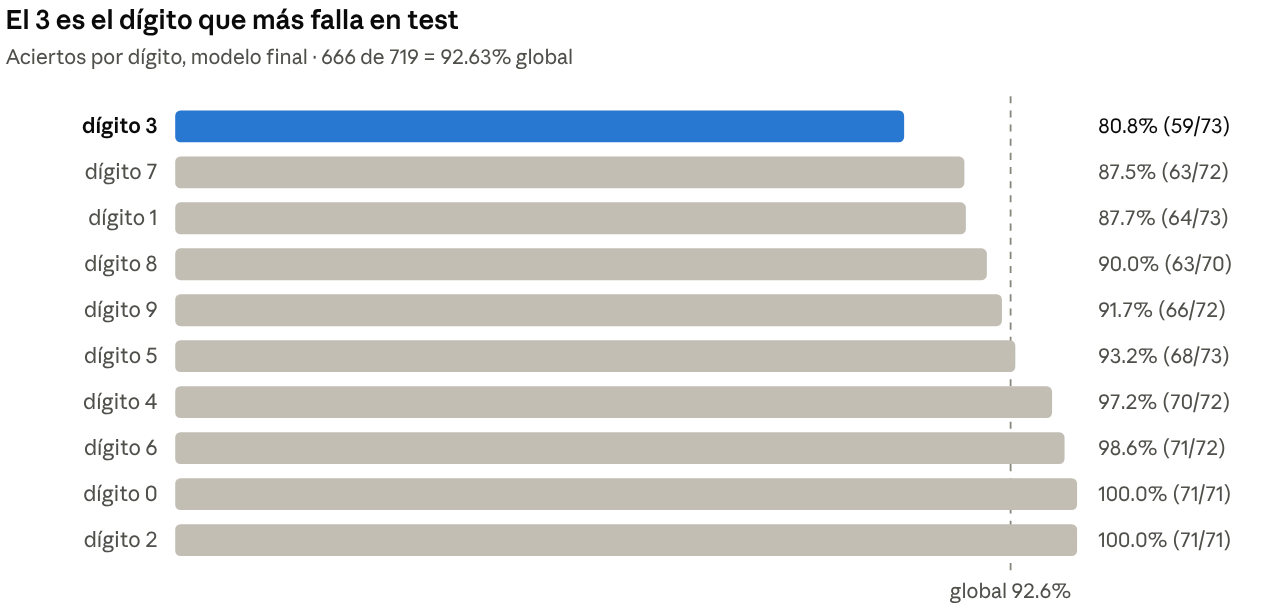
Con lr = 1e-2, monitoreando `val_loss` con paciencia 10, el modelo se detiene en la época 30, restaura la mejor (época 20) y llega a 92.63% en test. La diferencia frente al 41.86% de 5 épocas es solo más entrenamiento y una tasa de aprendizaje mayor.

Qué pasó en las 50 épocas



history del entrenamiento final · 30 épocas, reproducido con las mismas semillas que tu ejecución

La pérdida de entrenamiento sigue bajando hasta 0.01, pero la de validación se estanca alrededor de 0.21 a 0.25 desde la época 15: el modelo empieza a memorizar (sobreajuste). Como `EarlyStopping` monitorea `val_loss` con paciencia 10, al no mejorar entre las épocas 21 y 30 corta el entrenamiento y restaura los pesos de la época 20 (val_acc 0.958). Por eso nunca llega a la época 50.



predicciones del modelo final sobre las 719 imágenes de test · mismas semillas que tu ejecución

Con esos pesos el modelo acierta 666 de 719 imágenes de test. Los errores más frecuentes son 3 confundido con 9 (6 casos), 1 con 8 (7 casos) y 7 con 4 (5 casos): dígitos con trazos parecidos en una imagen de 8×8.

Parte 2: regresión (Diabetes)

Diabetes tiene 442 pacientes, 10 variables ya normalizadas y un objetivo continuo entre 25 y 346. Se reutiliza todo lo anterior cambiando tres cosas: salida lineal, pérdida MAE y métrica MAE.

2.1 define_regressor

```
def define_regressor(input_shape, hidden_layers, hidden_units, output_units):
    keras.utils.set_random_seed(0)
    keras.backend.clear_session()
    inputs = keras.layers.Input(shape=(input_shape,))
    x = inputs
    for _ in range(hidden_layers):
        x = keras.layers.Dense(hidden_units, activation='relu')(x)
    outputs = keras.layers.Dense(output_units, activation='linear')(x)
    model = keras.Model(inputs=inputs, outputs=outputs)
    return model
```

- Igual que `define_model`, pero con `activation='linear'`: la salida puede ser cualquier número real. Softmax o sigmoid la encerrarían entre 0 y 1 y nunca llegaría a 150.
- `clear_session()` reinicia los contadores de nombres, por eso las capas vuelven a llamarse `dense`, `dense_1`, `dense_2` como en la salida esperada.
- Parámetros: 10×8+8 = 88; 8×8+8 = 72; 8×1+1 = 9; total 169.

2.2 train_regressor

```
def train_regressor(model, filepath, X_train, y_train, epochs, batch_size,
                    learning_rate, validation_rate):
    model = compile_model(model, keras.losses.MeanAbsoluteError(),
                          keras.optimizers.Adam(learning_rate=learning_rate),
metrics=['mae'])
    callbacks = define_callbacks(filepath, 'val_loss', 'min', 10)
    history, model = train_model(model, X_train, y_train, epochs, batch_size,
                                 validation_rate, callbacks)
    return history, model
```

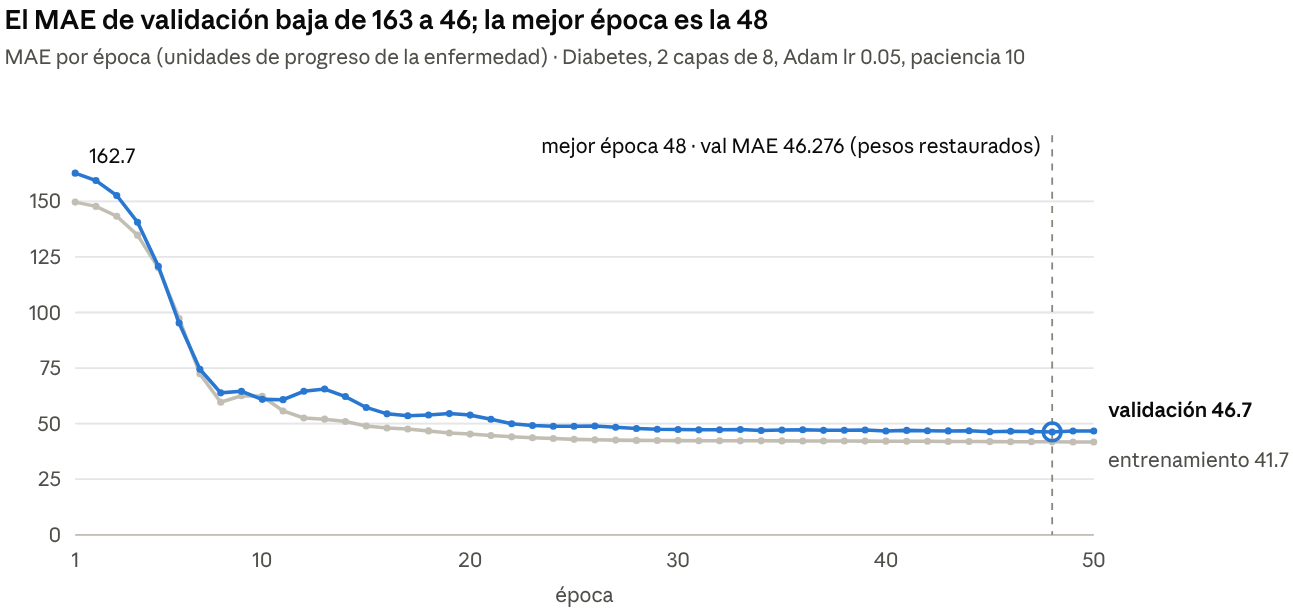
- Es la parte "incremental" de la tarea: no se reescribe nada, se componen `compile_model`, `define_callbacks` y `train_model`.
- Cumple cada nota: pérdida MAE, optimizador Adam con el `learning_rate` recibido, monitoreo de `val_loss` en modo `min` con paciencia 10, y métrica `'mae'` (por eso el log muestra `loss` y `mae` con el mismo valor).
- Las etiquetas van directo, sin one-hot: `y_train` ya es un número por paciente.

2.3 evaluate_regreesor

```
def evaluate_regreesor(model, X_test, y_test):
    loss, mae = model.evaluate(X_test, y_test)
    return mae
```

Mismo patrón que 1.6. El MAE de 44.67 significa que, en promedio, la predicción del progreso de la enfermedad se equivoca en unas 45 unidades sobre una escala de 25 a 346. El nombre `evaluate_regreesor` (con doble "e") se deja tal cual porque así lo busca el autograder.

Qué pasó en el entrenamiento del regresor



history de train_regressor · 50 épocas; época 1, mejor época y época 50 coinciden con tu salida en Colab

Aquí el early stopping nunca se activó: la validación seguía mejorando poco a poco (épocas 40, 45 y 48), así que la paciencia de 10 nunca se agotó y corrió las 50 épocas. Al final se restauraron los pesos de la época 48 (val MAE 46.276). La brecha entre entrenamiento (41.7) y validación (46.3) es pequeña, sin sobreajuste marcado.

Cómo leer el MAE de 44.67 en test:

Modelo	MAE en test
Predecir siempre el promedio de `y_train_2` (línea base)	62.98
Red neuronal 10 → 8 → 8 → 1 (la de la tarea)	44.67
Regresión lineal de scikit-learn, misma partición	43.46

La red reduce el error en 29% frente a la línea base, y queda muy cerca de una regresión lineal simple: en un dataset de 309 muestras y 10 variables una red pequeña no tiene mucho más que ganar. En test el objetivo va de 40 a 306, así que un error medio de unas 45 unidades es moderado.

Conceptos de fondo y errores comunes

La decisión clave es emparejar la salida de la red con el tipo de problema:

Problema	Activación de salida	Pérdida	Formato de etiquetas	Métrica
Binario	sigmoid (1 neurona)	BinaryCrossentropy	0/1	accuracy
Multiclase	softmax (c neuronas)	CategoricalCrossentropy	one-hot	accuracy
Multiclase, etiquetas enteras	softmax	SparseCategoricalCrossentropy	0..c-1	accuracy
Regresión	linear	MSE o MAE	número real	MAE

Softmax convierte las salidas z en probabilidades que suman 1:

```
\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{n} e^{z_j}}
```

Con z = [5, 3, 4] da [0.665, 0.090, 0.245], y la clase predicha es la de mayor probabilidad (argmax). La entropía cruzada categórica castiga con −log(p) la probabilidad asignada a la clase correcta: si la red da 0.665 a la clase verdadera, la pérdida es 0.41; si da 0.09, sube a 2.41.

MAE vs MSE: MAE = promedio de |y − ŷ| y se lee en las unidades del objetivo; MSE eleva al cuadrado y castiga más los errores grandes, por lo que es más sensible a valores atípicos.

Errores comunes:

- Usar `Sequential` cuando piden API funcional: el test 1_2 lo detecta.
- Olvidar `save_weights_only=True` o usar una ruta sin `.weights.h5`: Keras 3 lanza error.
- No poner `restore_best_weights=True`: el modelo final queda con los pesos de la última época, no de la mejor, y los números no coinciden.
- Llamar `define_model` sin re-ejecutar las celdas en orden: las semillas y el estado cambian y el accuracy difiere en decimales.

- Usar `metrics=['accuracy']` en vez de `['acc']`: funciona, pero el nombre en `history` cambia y los tests pueden fallar.

Observaciones de la ejecución

El entorno de Colab era más nuevo que el del enunciado, y aun así todos los números salieron iguales.

Librería	Versión del enunciado	Versión en tu ejecución
Python	3.11.12	3.13.16
NumPy	2.0.2	2.1.3
Keras	3.8.0	3.13.2
scikit-learn	sin dato	1.9.1 (instalada con `pip install -U`)

Diferencias visibles respecto a las salidas esperadas, ninguna afecta la nota:

- **"loss improved from None"**: Keras 3.13 imprime `None` donde 3.8 imprimía `inf` para la primera época. Es solo el mensaje; el valor guardado es el mismo.
- **`y_test_2.min()` da 40.0, no 25.0**: el enunciado espera `(309, 10), (133, 10), 346.0, 25.0`, pero el 25 es el mínimo de todo el dataset y cae en la partición de *entrenamiento*. En test el rango real es 40 a 306. Es un error del texto esperado, no del código (la celda no es calificada).
- **Mensajes "finished saving model"**: Keras 3.13 agrega esa línea tras cada guardado del checkpoint. Es informativa.
- **"No se pueden guardar cambios"**: el notebook está abierto en modo solo lectura. El envío ya quedó registrado en UNCode, pero si quieres conservar el código usa Archivo → Guardar una copia en Drive.

Por qué los números son reproducibles: `keras.utils.set_random_seed(0)` se llama al crear el modelo (fija los pesos iniciales) y otra vez justo antes de `fit` (fija el barajado de batches). `validation_split` siempre toma el último 20% de las filas sin barajar, así que la partición de validación también es fija.

Entrega

La tarea está entregada: `grader.submit_task(globals())` corrió en aproximadamente 1 minuto y devolvió "Your submission grade: 100.0%".

- [ ] Revisar la submission en la página de la tarea en UNCode.
- [ ] Guardar una copia del notebook en Drive (Archivo → Guardar una copia en Drive) para conservar el código.

Si alguna vez hay que volver a correrlo: Entorno de ejecución → Reiniciar sesión y luego Ejecutar todas, en orden. Las semillas dependen de ese orden; ejecutar celdas sueltas puede mover los decimales y hacer fallar un test.